

Project Overview

VisionX - Home Security System

Problem

Home security cameras are expensive, especially with subscriptions, and are difficult to customize. I want a system that allows me to monitor cameras, save footage locally to my NAS, and add custom automation features like facial recognition without relying on third-party services.

Many security camera subscription services make it difficult to cancel the subscription, so I have just been getting charged for simple camera functions that aren't fully used. I wanted to build something that can be run on your own device and is scaled based on available hardware, making it fully customizable and cheap.

Overview

This project is a self-hosted home security camera platform with a web dashboard and mobile app for viewing live camera feeds, storing footage on a NAS, and monitoring device health. This system will eventually support motion detection, facial recognition, alerts, mmWave heartbeat detection, and customizable automations.

Goal

The goal is to build a working MVP that can connect to at least one IP camera, display a live feed in a web dashboard, save recordings to my NAS, and show basic camera status such as online/offline health.

Target Users

The first user is myself, for monitoring my home. Future users could include people who want a customizable, self-hosted alternative to a subscription-based security service

MVP Scope

- Web dashboard to view live camera feeds
- Footage stored on NAS
- Device health monitoring
- Basic authentication for the dashboard
- Add or manage one or more camera streams

Out of Scope for MVP

- Mobile app
- Motion Detection
- Facial Recognition
- Notification Rules
- Multi-user permissions
- Cloud backup

Available Devices

- Pi 5 with Hailo Inference Chip
- Arducam
- Dell Optiplex 8500T 8GB RAM
- mmWave sensors
- ESP32

Success Criteria

- I can open the dashboard and view the live camera feed
- System detects when camera goes offline
- Recordings saved to NAS
- App can run continuously on homelab using docker

Tech Stack

- Frontend Web: React + Vite + Tailwind CSS
- Backend API: FastAPI + Python
- Database: PostgreSQL
- Live Streaming: go2rtc
- Recording: FFmpeg
- Storage: NAS mounted into backend/worker containers
- Background Jobs: Redis + Python workers
- Deployment: Docker Compose
- Reverse Proxy: Caddy
- Remote Access: Tailscale
- Monitoring: Prometheus + Grafana
- Mobile: PWA first, React Native/Expo later
- AI: OpenCV first, ONNX/TensorFlow Lite later

Database

- Users
- Cameras
- Recordings (file path)
- Motion events
- Face detections
- Recognized people
- Device health
- Alerts
- Automation rules
- Camera status logs

Ex.

recording_id: 123

camera_id: front_door

start_time: 2026-06-27 10:00:00
end_time: 2026-06-27 10:05:00
file_path: /mnt/nas/security/front_door/2026-06-27/10-00.mp4
has_motion: true
has_person: true

Tradeoffs

- FastAPI because AI/video processing mainly uses python

Considerations

Camera Streaming: Frigate vs go2rtc vs build myself

Frigate

- Open source local NVR designed for IP cameras with AI object detection
- Uses OpenCV and TensorFlow for local object detection
- Recommends using a dedicated detector instead of CPU detection
- Supports:
 - Camera streams, recording, snapshots, object detection, MQTT events, Home Assistant integration, Retention rules, local processing
- Drawbacks:
 - Custom facial recognition may require a separate service
 - App may depend heavily on Frigate's design

Go2rtc

- Takes camera streams and restream through RTSP/webRTC
- Helps with low-latency live viewing and reducing direct connections to cameras
- Drawbacks:
 - Not full NVR by itself
 - Need to build recording, detection, events, storage, and UI

Build everything with FFmpeg/OpenCV

- Pros:
 - Full control recording, detection, and storage logic
- Drawbacks:
 - Video streaming difficult to program
 - RTSP issues are common
 - Browser live playback is not trivial
 - Recording reliability is hard
 - Mobile streaming becomes more difficult to implement

Database

TimescaleDB

- PostgreSQL extension for time-series data through hypertables, partitioned by time for analytics workloads
- Useful if storing lots of camera health logs, motion counts, object counts, temperature readings, heartbeat data, or metrics over time

Mobile App

Progressive Web App (PWA)

- Build dashboard as mobile-friendly web app, host it on homelab, and open in Safari and add to homescreen
- Pros:
 - No app store or apple developer account required
 - No testflight
 - No native IOS signing
 - Easy to update
- Drawbacks:
 - Not as native-feeling as real IOS app
 - Limited access to iphone APIs

- Push notifications are more complicated than native apps
- Background tasks are limited

Expo / React Native

- Pros:
 - Real app
 - Better native navigation
 - Better access to device APIs
 - Better push notification
- Drawbacks:
 - More setup and requires Apple signing/provisioning
 - Requires paid Apple Developer account
 - More difficult updates
 - Separate mobile codebase from web dashboard

Architecture

IP Camera

↓ RTSP

go2rtc

↓

Live stream for browser/PWA

IP Camera

↓ RTSP

FFmpeg worker

↓

NAS .mp4 recording files

↓

PostgreSQL recording metadata

FastAPI



Coordinates cameras, users, recordings, health, events, alerts

React Dashboard / PWA



Calls FastAPI APIs



Views live feeds, recordings, camera status, events

AI Workers



Analyze frames/clips



Create motion/person/face events

Milestones

Phase 1: Build MVP

Phase 1.1: Research camera streaming, tech stacks, RTSP

Phase 1.2: Build backend camera connection prototype

Phase 1.3: Build web dashboard live view

Phase 1.4: Add local/NAS recording

Phase 1.5: Dockerize and deploy

Phase 1.6: Polish README and resume write-up

Fun Possible Additions

- Object tracking for intruders

Project MVP

Tech Stack

- React + Vite + Tailwind
- FastAPI
- PostgreSQL
- Docker Compose
- go2rtc
- FFmpeg
- NAS mount
- Redis
- Basic auth
- Camera health checks
- Recording metadata
- PWA-friendly web dashboard

Docker Compose Services

- Visionx-web
 - Login page, camera grid, live camera view, recording timeline, camera health page, events page, settings page
- Visionx-api
 - Authentication, camera CRUD, recording metadata, event metadata, health status, alert rules, serving recording URLs, starting/stopping workers indirectly
- Visionx-worker
 - Python background worker
 - Running FFmpeg recording jobs, checking camera health, creating recording rows, generating thumbnails, running motion/object detection, cleaning up old recordings, sending alerts
- Visionx-postgres
 - Stores metadata
- visionx-redis
- Visionx-go2rtc
 - Connecting to RTSP cameras
 - Restreaming to browser-compatible formats

- Reducing direct camera connections
 - Providing live stream endpoints
- visionx-caddy
- visionx-prometheus
- Visionx-grafana

Project Folder Structure

```
visionx/  
  frontend/  
    src/  
      pages/  
      components/  
      api/  
      hooks/  
      types/  
    vite.config.ts  
    tailwind.config.js  
  
  backend/  
    app/  
      main.py  
      config.py  
      database.py  
      models/  
      schemas/  
      routers/  
      services/  
      workers/  
      auth/  
    alembic/  
    pyproject.toml  
  
  docker/  
    go2rtc/  
      go2rtc.yaml  
    caddy/  
      Caddyfile
```

```
storage/
  recordings/
  snapshots/
  thumbnails/

docker-compose.yml
README.md
```

Database Design

Tables:

- **Users**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid()
 - email TEXT UNIQUE NOT NULL
 - password_hash TEXT NOT NULL
 - display_name TEXT
 - role TEXT NOT NULL DEFAULT 'admin'
 - created_at TIMESTAMPTZ NOT NULL DEFAULT now()
 - updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
- **Cameras**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 - name TEXT NOT NULL,
 - location TEXT,
 - rtsp_url TEXT NOT NULL,
 - stream_key TEXT UNIQUE NOT NULL,
 - enabled BOOLEAN NOT NULL DEFAULT true,
 - recording_enabled BOOLEAN NOT NULL DEFAULT true,
 - health_check_enabled BOOLEAN NOT NULL DEFAULT true,
 - created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 - updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
- **Recordings**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 - camera_id UUID NOT NULL REFERENCES cameras(id) ON DELETE CASCADE,
 - start_time TIMESTAMPTZ NOT NULL,
 - end_time TIMESTAMPTZ,
 - duration_seconds INTEGER,
 - file_path TEXT NOT NULL,

- file_size_bytes BIGINT,
- format TEXT NOT NULL DEFAULT 'mp4',
- status TEXT NOT NULL DEFAULT 'recording',
- has_motion BOOLEAN NOT NULL DEFAULT false,
- has_person BOOLEAN NOT NULL DEFAULT false,
- created_at TIMESTAMPTZ NOT NULL DEFAULT now()
- **Camera_health_checks**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 - camera_id UUID NOT NULL REFERENCES cameras(id) ON DELETE CASCADE,
 - checked_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 - status TEXT NOT NULL,
 - latency_ms INTEGER,
 - last_frame_at TIMESTAMPTZ,
 - last_recording_at TIMESTAMPTZ,
 - error_message TEXT
- **Events**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 - camera_id UUID NOT NULL REFERENCES cameras(id) ON DELETE CASCADE,
 - recording_id UUID REFERENCES recordings(id) ON DELETE SET NULL,
 - event_type TEXT NOT NULL,
 - start_time TIMESTAMPTZ NOT NULL,
 - end_time TIMESTAMPTZ,
 - confidence NUMERIC(5, 4),
 - label TEXT,
 - description TEXT,
 - snapshot_path TEXT,
 - metadata JSONB,
 - created_at TIMESTAMPTZ NOT NULL DEFAULT now()
- **Snapshots**
 - id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 - camera_id UUID NOT NULL REFERENCES cameras(id) ON DELETE CASCADE,
 - event_id UUID REFERENCES events(id) ON DELETE SET NULL,
 - recording_id UUID REFERENCES recordings(id) ON DELETE SET NULL,
 - captured_at TIMESTAMPTZ NOT NULL,
 - file_path TEXT NOT NULL,

- file_size_bytes BIGINT,
- created_at TIMESTAMPTZ NOT NULL DEFAULT now()

ERD Relationship

